Stat Arb for Non Quant · เล่ม 1 — เชือกที่ผูกสองราคา

ภาคผนวก I — สำหรับคนเขียนโค้ด

ตัวเลขทั้งเล่มสร้างอย่างไร รันซ้ำอย่างไร การ์ดกติกาเขียนเป็นโค้ดเทียมได้แบบไหน และอ่านงานวิจัยคนอื่นโดยเริ่มจากบัญชี

🧭 **หน้านี้สำหรับใคร**

เล่มนี้ตั้งใจให้ทำได้ครบโดยไม่เขียนโค้ด (ตาราง ใบงาน และ sab-workbook.xlsx แทนโค้ด) หน้านี้สำหรับคนที่เขียนโค้ดเป็นและอยากรู้สามเรื่อง: ตัวเลขทั้งเล่มสร้างและตรวจอย่างไร · การ์ดกติกาของบท 12 เขียนเป็นโค้ดเทียมได้แบบไหน · อ่านงานวิจัยหรือ tutorial ของคนอื่นโดยไม่หลงตัวเลขสวย

โค้ดในหน้านี้เป็น**โค้ดเทียม**ที่ใช้แค่ลูปและตัวแปรธรรมดา และไม่อ้างไลบรารีหรือ API ของ exchange ใด

1 · ตัวเลขทั้งเล่มมาจากสคริปต์เดียว

ทุกตัวเลขในบทอยู่ใน `docs/sab-figures.json` ซึ่งสร้างโดย `tools/sab_figures.py` ไฟล์เดียว สคริปต์นี้ไม่ต่อเน็ต ไม่ใช้ข้อมูลจริง และได้ผลเดิมทุกครั้งที่รัน

- **ใช้ไลบรารีมาตรฐานของ Python ล้วน** คิดด้วย `Decimal` แล้วปัดครึ่งขึ้นตามจำนวนทศนิยมที่บทพิมพ์ ส่วนค่าจำลองใช้ตัวสุ่มของ Python ด้วย seed ตายตัว (seed ทั้งหมดอยู่ใน ภาคผนวก H)
- **ตรรกะพอร์ตมาจากสคริปต์ของต้นฉบับ v0.9** ที่ `sources/sab-v0_9/code/` แต่ตอนสร้างตัวเลขไม่ได้ import โค้ดต้นฉบับ โค้ดต้นฉบับถูกโหลดเฉพาะตอนตรวจเทียบเท่านั้น
- **ค่าใหม่ของแต่ละบท** อยู่ในปลั๊กอิน `tools/sab_figures_ch/chNN.py` หนึ่งไฟล์ต่อบท แต่ละไฟล์มีฟังก์ชัน `extend(data, D, q)` ที่คืนคีย์ใหม่ไปลงใต้ `chNN` โดยห้ามทับคีย์เดิม ส่วนข้ออ้างเชิงตลาดของบทเข้าทะเบียน `_claims` ผ่านปลั๊กอินเดียวกัน
- **คีย์เป็นภาษาไทย** เช่น `ch04.ต้องโอน` ส่วนค่าเต็มความละเอียดของต้นฉบับเก็บแยกไว้ใต้ `_ต้นฉบับ`

รันซ้ำและตรวจ

```
# สร้าง JSON ใหม่ทั้งไฟล์
python3 -I tools/sab_figures.py

# คำนวณใหม่แล้วเทียบกับไฟล์ที่มีอยู่ (ไม่เขียน) — ต่างแม้ตัวเดียว = ไม่ผ่าน
python3 -I tools/sab_figures.py --check

# เทียบกับโค้ดต้นฉบับ v0.9 ทีละค่าตามแผนที่ tools/sab_origmap.json
# (ต้องมีโฟลเดอร์ต้นฉบับ · ตั้งตำแหน่งอื่นด้วยตัวแปร SAB_SOURCE_DIR)
python3 -I tools/sab_figures.py --check-original

# เติมค่าจาก JSON ลงหน้าเว็บ แล้วตรวจว่าทุกตัวเลขในหน้ามีใน JSON
python3 tools/sab_charts.py --fill --scope part4
python3 tools/sab_check_figures.py --scope part4

# สมุดงาน: สร้าง xlsx แล้วให้ LibreOffice คำนวณสูตรจริงเทียบกับ JSON
python3 tools/sab_workbook.py --check
```

กฎเหล็กของเล่มคือ**ห้ามมีตัวเลขลอย**: ตัวตรวจอ่านทุกตัวเลขที่ผู้อ่านเห็นในหน้า ถ้าไม่มีใน JSON ถือว่าผิด (ยกเว้นจำนวนนับเล็ก ๆ) ส่วนการตรวจเทียบต้นฉบับผ่านครบทุกค่าที่จับคู่ได้ และค่าที่ไม่ใช้ในเล่มมีเหตุผลกำกับไว้ใน `tools/sab_origmap.json`

บทบาทของโค้ดต้นฉบับ

ต้นฉบับ v0.9 มีสคริปต์ 10 ไฟล์ (8 ไฟล์คำนวณ + 2 ไฟล์วาดภาพที่ต้องใช้ matplotlib) และผลลัพธ์เป็น JSON 8 ไฟล์ ซึ่งใช้ตรวจหน่วย บัญชี และตรรกะของตัวอย่างสมมติ โดยไม่ดาวน์โหลดข้อมูลตลาดและไม่ส่งคำสั่ง แต่ตารางโค้ดในต้นฉบับผูกสคริปต์กับบทผิดบางแถว และคำสั่งรันสร้าง JSON บางไฟล์ไม่ได้ (errata ข้อ 28) เล่มนี้จึงไม่พิมพ์ตารางนั้นซ้ำ แต่ใช้สคริปต์เป็น**ต้นแบบที่ต้องได้ค่าตรงกัน**เท่านั้น

ไฟล์ต้นฉบับ	ตรวจเรื่องอะไร	ไปอยู่ในบทไหนของเล่มนี้
`v07_primer_checks.py`	long/short · leverage · ต้นทุนหนึ่งรอบ · funding	0 · 1
`signal_design_demo.py`	อัตรา basis · ตะกร้าคงที่ · z เปลี่ยนโดยราคาไม่เปลี่ยน	3 · 5 · 9
`ch03_ledger_demo.py`	กระทบยอดราคา เงินสด equity · ปรับ hedge · ต้นทุน · ได้ของไม่ครบ	3 · 11 · 16
`part1_reasoning_demo.py`	แยกผลที่คาดออกจากผลจริงทีละชั้น · exposure	8 · 15
`methods_signals_stops_demo.py`	ผลออกสามตอนจบ · โหมดพักต้องกลับมาอย่างตั้งใจ	12 · 14
`mean_reversion_demo.py`	ค่าเฉลี่ยภายใต้ AR(1) · ครึ่งชีวิต · ค่ากลางที่เลื่อน	13 · 14
`v06_enrichment_demo.py`	รายรับสม่ำเสมอกับความเสียหายใหญ่ · บัญชีหลักประกัน · เทียบ hedge · จำลองเวลาแตะเป้า	2 · 4 · 8 · 13
`v09_ch06_ch07_demo.py`	ผลต่อทุนของ cash-and-carry · ปิดก่อนหมดอายุ · สี่เส้นทาง funding · คู่จำลอง 200 คู่	5 · 6 · 7 · 9 · 10

ชื่อไฟล์และหน้าที่ตามส่วน "โค้ดคำนวณประกอบบท" ของต้นฉบับ · คอลัมน์ขวาคือบทของเล่มนี้ที่ใช้ค่าจากตรรกะนั้น (ดู `tools/sab_origmap.json` สำหรับการจับคู่ทีละค่า)

2 · บัญชีตามเหตุการณ์ (วงหลักของทุก backtest)

ลูปนี้ดัดแปลงจากโค้ดเทียมในต้นฉบับ: กำไรของช่วงคิดจากของที่ถืออยู่*ก่อน*ราคาขยับ และของที่ถือจะเปลี่ยนเมื่อซื้อขายได้จริงเท่านั้น ไม่ใช่ตอนส่งคำสั่ง (สูตรเดียวกันอยู่ใน ภาคผนวก F หัวข้อ 5)

```
# เริ่มด้วยราคาที่ใช้ได้และบัญชีเปิดที่บันทึกไว้
for event in events_sorted_by_time:
    if event.is_market_update:
        update_marks(event)                       # ราคาประเมินล่าสุดของแต่ละขา

    if event.is_fill:                             # ซื้อขายได้จริง (อาจได้ไม่ครบ)
        cash -= event.signed_qty * event.fill_price
        cash -= event.fee
        held[event.asset] += event.signed_qty     # ซื้อ = บวก · ขาย = ลบ
        update_pending_orders(event)

    if event.is_cashflow:                         # funding · ค่าถือ (มีเครื่องหมาย)
        cash += event.signed_amount

    equity = cash + sum(held[i] * mark[i] for i in assets)

    if event.is_signal_time:
        signal = model(data_known_by(event.time))   # ใช้เฉพาะข้อมูลที่รู้แล้ว ณ เวลานั้น
        target = size_positions(signal)
        reconcile_and_submit(target, held, pending_orders)   # หักคำสั่งค้างก่อน ไม่สั่งซ้ำ
```

- การส่งคำสั่งยังไม่เปลี่ยน `held` ส่วนการได้ขาเดียว (บท 16) คือสถานะที่ `held` ไม่เท่า `target`
- ราคาที่แย่กว่าราคาประเมินอยู่ในบรรทัด `fill_price` แล้ว อย่าหัก slippage ซ้ำ
- เหตุการณ์เวลาเดียวกันต้องกำหนดลำดับย่อยให้ชัด: ราคาใดพร้อมให้โมเดลใช้ก่อนส่งคำสั่ง และ funding คิดจากสถานะ ณ เวลาใด

3 · การ์ดกติกา 6 บรรทัดของบท 12 เป็นโค้ดเทียม

การ์ดของมิน (คู่ A–B · X = A − 2B · ไม้บรรทัดค่ากลาง 0 ระยะแกว่ง 1 จึงอ่าน X เป็น z ได้ตรง) เข้าเมื่อ X ≥ 2 · ออกเมื่อ X ≤ 0.5 · ถือไม่เกิน 3 ช่วง · งบขาดทุนสุทธิ −15 · ต้นทุน 0.4 + 0.6 ต่อไม้ · ลำดับเมื่อหลายเหตุพร้อมกัน ขาดทุน → สัญญาณ → เวลา · ออกเพราะงบแล้วหยุดเปิดไม้ใหม่

```
# การ์ดกติกาคู่ A–B (บท 12) — โค้ดเทียม · ค่าของเล่น ไม่ใช่คำแนะนำ
ENTRY, EXIT, MAX_HOLD = 2, 0.5, 3      # ช่วง = เปิดดู X หนึ่งครั้ง
BUDGET = -15                            # งบขาดทุนสุทธิ รวมต้นทุน
COST = 0.4 + 0.6                        # เข้า + ออก ต่อไม้
SETS = 10                               # 1 ชุด = A 1 เหรียญ คู่ B 2 เหรียญ

mode, pos = "ACTIVE", None
for t, x in enumerate(spread_series):   # x = A - 2 * B ณ ช่วง t
    if pos is None:
        if mode == "PAUSED":
            continue                    # บท 14: ไม่เปิดไม้ใหม่ แต่ยังดูแลของที่ถือ
        if x >= ENTRY:
            pos = open_pair(side=-1, x_in=x, t_in=t)   # short X: ขาย A 10 (ผ่าน perp) ซื้อ B
        elif x <= -ENTRY:
            pos = open_pair(side=+1, x_in=x, t_in=t)   # long X: กลับด้าน
        continue

    net_now = SETS * pos.side * (x - pos.x_in) - COST  # ถ้าปิดตอนนี้ได้สุทธิเท่าไร
    if net_now <= BUDGET:                   # 1) ขาดทุนมาก่อน
        close_pair(pos, reason="งบ")
        mode = "PAUSED"                     #    ออกเพราะงบ = หยุดเปิดไม้ใหม่
    elif -pos.side * x <= EXIT:             # 2) สัญญาณ: short ออกเมื่อ x <= 0.5 · long เมื่อ x
        close_pair(pos, reason="เป้า")
    elif t - pos.t_in >= MAX_HOLD:          # 3) เวลา
        close_pair(pos, reason="เวลา")
    if pos.closed:
        pos = None
```

ตรวจกับสามเส้นทางที่บทซ้อม (เข้าที่ 2.1 ทุกเส้น): 2.1 → 1.7 → 0.4 ออกเพราะเป้า ได้ 16 · 2.1 → 2.0 → 1.9 → 1.8 ออกเพราะเวลา ได้ 2 · 2.1 → 2.8 → 4.1 ออกเพราะงบ ได้ −21 · เส้นทางที่ 4 ครบเวลาและชนงบในช่วงเดียวกัน งบชนะตามลำดับ ได้ −17

⚠️ **สามจุดที่โค้ดแบบนี้หลอกได้**

- **ราคาที่ปิดได้ไม่ใช่ราคาที่เห็น:** โค้ดเช็กงบที่ X ที่สังเกต แต่ X กระโดดจาก 2.8 ไป 4.1 โดยไม่มีจุดให้ซื้อขายระหว่างทาง งบ −15 จึงกลายเป็น −21 ถ้า backtest ปิดที่เส้นงบพอดีทั้งที่ข้อมูลกระโดด แปลว่ามันกำลังหลอก
- **สัญญาณกับการกด:** เรารู้ x ของช่วง t หลังราคาปิดแล้ว คำสั่งจึงได้ราคาของช่วงถัดไป ถ้าโค้ดเข้าที่ x เดียวกับที่ใช้ตัดสิน แปลว่าโค้ดกำลังใช้ข้อมูลอนาคต
- **ค่าถือและ beta:** โค้ดนี้ยังไม่มีค่าถือขา short (funding) และคู่นี้ไม่เป็นกลางต่อตลาด (beta exposure +500 · บท 8) ผลจึงยังไม่ใช่ผลจริง และถ้าใช้คำสั่งซื้อทันที ต้นทุนจริงจะราว 2 ต่อไม้ (บท 16)

4 · อ่านงานวิจัยหรือ tutorial ของคนอื่น: เริ่มจากบัญชี

ก่อนดู Sharpe หรือกำไรสะสมทั้งปี ให้ตอบสามแกนนี้ให้ได้ (จากหัวข้อ "ใช้บัญชีนี้แกะกรณีศึกษาของผู้อื่น" ของต้นฉบับ)

แกน	คำถามที่ต้องตอบ
เลือกสินทรัพย์ (asset selection)	ทำไมสองตัวนี้อยู่ด้วยกัน (เชือกชนิดไหน) · ราคาและหน่วยเข้ากันไหม · แต่ละขาต้องซื้อขายแบบใด (short ได้ไหม ค่ายืมหรือ funding เท่าไร)
ขั้นตอน (workflow)	ข้อมูลใดเข้าประมาณ h · สัญญาณเกิดเมื่อไร · ข้อมูลส่วนไหนยังไม่ควรถูกใช้ ณ ตอนนั้น · ช่วงเลือกกับช่วงทดสอบแยกกันไหม · ค้นมากี่คู่
จัดการสถานะ (position management)	h แปลงเป็นจำนวนอย่างไร · เปิดหรือปรับเมื่อไร · fill ต้นทุน เงินสด หลักประกันอยู่ตรงไหนในบัญชี

1. **คิดบัญชีด้วยมือหนึ่งช่วงสั้น ๆ** (ราคา · จำนวนที่ถือ · เงินสด · equity) ถ้าไม่ตรงกับโปรแกรม ให้หยุดไล่ช่วงนั้นก่อน สาเหตุที่พบบ่อยคือเครื่องหมายของขา short ผิด ใช้จำนวนผิดช่วงเวลา หรือหักต้นทุนซ้ำ
2. **หาตัวแปรที่เก็บจำนวนซื้อขายจริง** และดูว่า "spread" ในงานนั้นคือราคา ผลตอบแทน หรือลอการิทึมของราคา และกำไรคิดด้วยจำนวนของช่วงไหน
3. **แยกสิ่งที่ประมาณจากข้อมูลกับสิ่งที่ผู้เขียนเลือก** เช่น เส้นเข้าออก จำนวนสินทรัพย์ และความเร็วปรับกลับที่ตั้งคงที่ แล้วดูว่างานนั้นทำอะไรเมื่อไม่มีคู่ผ่าน และจัดการสถานะค้างอย่างไร
4. **จดสิ่งที่เปลี่ยนเมื่อย้ายตลาด** เช่น จากหุ้นเป็น perp ต้องเพิ่มชนิดสัญญา ตัวคูณสัญญา funding และหลักประกัน ถ้าจะใช้คำสั่งตั้งรอเท่านั้น ต้องเปลี่ยนสมมติฐานเรื่องคำสั่งได้ราคาทั้งหมด การคงชื่อโมเดลเดิมไม่ได้แปลว่าบัญชีเหมือนเดิม
5. **ติดระดับหลักฐานให้ข้อสรุป** อ่านวิธีแล้ว · ตรวจโค้ดแล้ว · รันตัวอย่างแล้ว · ทำซ้ำผลได้แล้ว — สี่ระดับนี้ต่างกันมาก และกรณีศึกษาทั้งหมดในต้นฉบับไม่มีชุดใดถึงระดับทำซ้ำผล

คำถามระดับสมมติฐาน (ทำไม spread ควรกลับ · คัดคู่ก่อนหรือหลังเห็นผล) ค่อยถามหลังบัญชีผ่านแล้ว ส่วนใบอ่าน backtest 7 แถวที่ไม่ต้องอ่านโค้ดอยู่ใน บท 18

แหล่งโค้ดที่ต้นฉบับอ้าง (ยังไม่ได้ตรวจ)

ต้นฉบับอ้างเอกสารของแพลตฟอร์ม backtest และไลบรารีพอร์ตหลายรายการ เช่น เอกสาร fill และ order event ของ QuantConnect (R2 · R48) · ไฟล์จัดการ drawdown ใน LEAN (R45) · ตัวอย่างกลยุทธ์เป็นกลางต่อตลาดของ Cvxportfolio (R35) · implementation ของ ArbitrageLab (R11 · R43) เล่มนี้**ไม่ได้รันหรือเปิดตรวจแหล่งเหล่านี้** และไม่สอนการใช้ไลบรารีใด รายการพร้อมสถานะอยู่ใน ภาคผนวก H

🔗 **ต่อจากหน้านี้**

- สูตรเบื้องหลังตาราง → ภาคผนวก F · live กับ backtest ไม่ตรงกัน → statarb-live-vs-backtest
- ข้อมูลทดสอบเชื่อได้แค่ไหน → statarb-data-quality · ทดสอบหลายครั้งจนเจอของปลอม → math-part11 หัวข้อ Multiple Testing